

Real-Time Operating Systems

for IoT & Embedded Systems

RTOS Concepts · FreeRTOS Architecture · Tasks, Scheduler, Queues & Semaphores · ESP32 Demonstration

What We'll Cover

1

What is an RTOS?

Definition, real-time guarantees, and why FreeRTOS is preferred over a general-purpose OS on constrained devices.

2

Where FreeRTOS fits

Comparing Arduino, ESP-IDF, MicroPython, and FreeRTOS-based bare-metal as IoT programming frameworks.

3

FreeRTOS architecture

Tasks, the scheduler, queues, and semaphores — and how they enable real-time multitasking on the ESP32.

4

Microcontroller vs SBC

Why an RTOS-based MCU (ESP32/AVR) and a Linux-based SBC (Raspberry Pi) solve different problems.

· CORE CONCEPT

What Is a Real-Time Operating System?

An RTOS is an operating system designed to process inputs and produce a response within a **guaranteed, bounded time — a deadline.**

KEY IDEA: CORRECTNESS DEPENDS ON TIME

A late answer is a wrong answer — not just a slow one. This is what separates an RTOS from a general-purpose OS optimized for average throughput.

HARD REAL-TIME

Missing a deadline = system failure.
e.g. airbag deployment, anti-lock brakes, pacemaker timing.

SOFT REAL-TIME

Missing a deadline degrades quality, but the system survives.
e.g. video streaming, an IoT sensor sending a reading a little late.

Most IoT nodes (sensing → processing → transmission) are soft real-time systems — which is exactly the class FreeRTOS targets.

RTOS vs. General-Purpose OS

Aspect	RTOS (e.g., FreeRTOS)	General-Purpose OS
Scheduling goal	Meet deadlines — determinism	Maximize average throughput / fairness
Scheduler	Priority-based, preemptive, predictable	Time-sliced, best-effort, variable latency
Memory footprint	A few KB – hundreds of KB RAM	Tens to hundreds of MB RAM
Boot time	Milliseconds	Seconds to tens of seconds
Typical hardware	Microcontrollers (ESP32, AVR, ARM Cortex-M)	SoCs / SBCs (Raspberry Pi, PC)
Examples	FreeRTOS, Zephyr, RIOT, VxWorks	Linux, Windows, Android

Why Constrained IoT Devices Need an RTOS

1

Determinism

Sensor sampling, control loops, and radio timing (e.g. BLE/Zigbee slots) need predictable response — not just fast, but on-time.

2

Tiny footprint

Runs in a few KB of RAM/flash — fits microcontrollers that could never boot Linux.

3

Concurrency without complexity

Lets one MCU juggle sensing, processing, and networking as independent tasks instead of one tangled loop.

4

Power efficiency

Built-in idle/tickless modes sleep the CPU between events — critical for battery-powered motes.

RTOS Options in the IoT World

FreeRTOS

Free, open-source, tiny kernel. De-facto standard on ESP32, STM32, AVR.

Zephyr

Linux-Foundation project. Broader hardware abstraction, built-in networking stack.

RIOT OS

Academic/research roots. Strong 6LoWPAN & CoAP support for constrained mesh nodes.

Mbed OS

ARM's RTOS for Cortex-M boards, with a large driver/library ecosystem.

VxWorks

Commercial, safety-certified. Used in aerospace, industrial, and automotive.

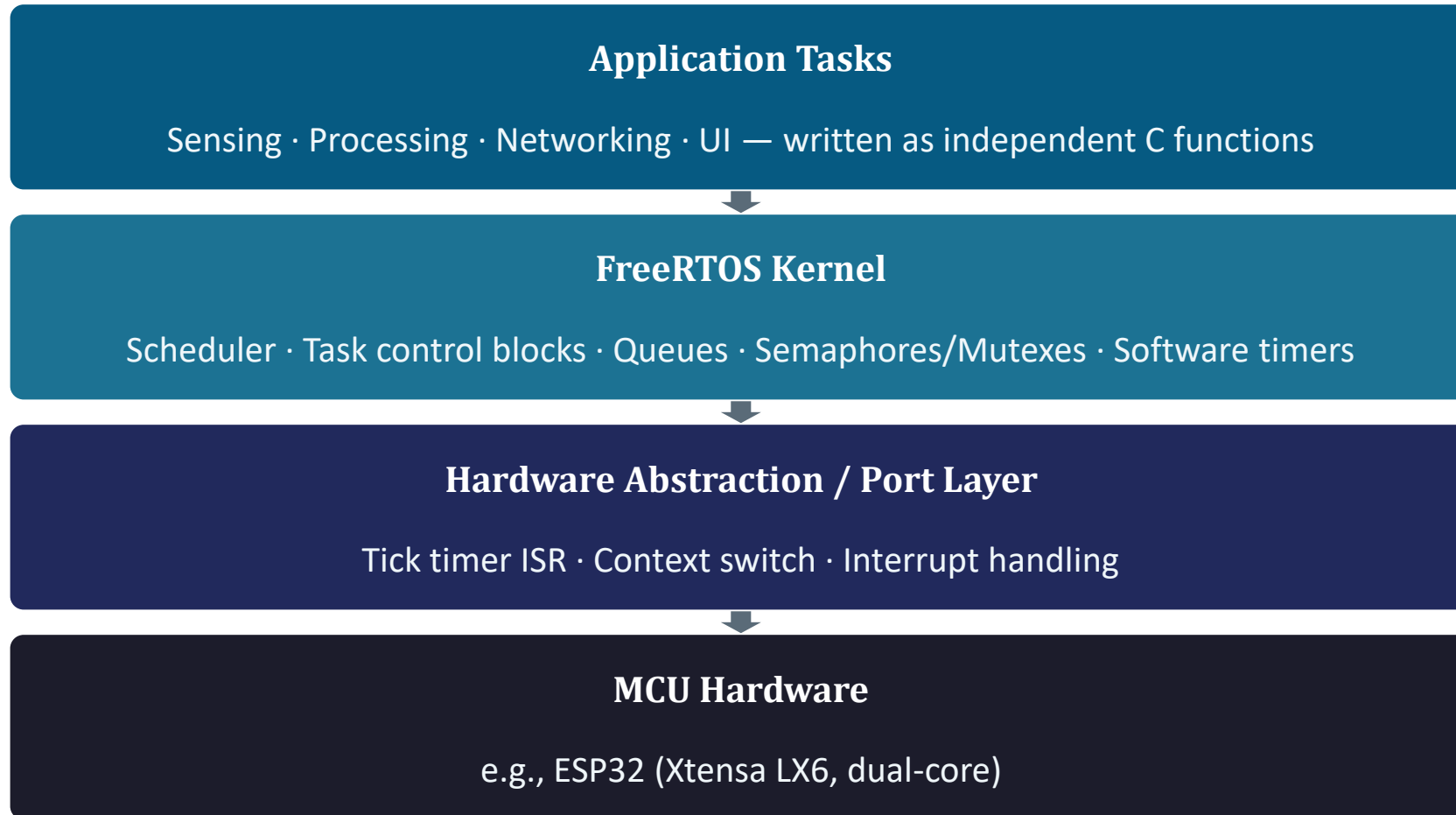
This course focuses on FreeRTOS — it ships built-in with ESP-IDF, the framework used for our ESP32 labs.

Why FreeRTOS Is Preferred for Constrained Devices

- **Free & open source (MIT license)** No licensing cost; source is auditable and modifiable.
- **Minimal footprint** Kernel can fit in ~6–12 KB of flash — leaves room for the application.
- **Broad MCU support** Ported to 40+ architectures: ESP32, ARM Cortex-M, AVR, PIC, RISC-V.
- **Industry backing** Maintained by Amazon Web Services; integrates directly with AWS IoT Core.
- **Mature ecosystem** Native to ESP-IDF; huge community, documentation, and reference examples.

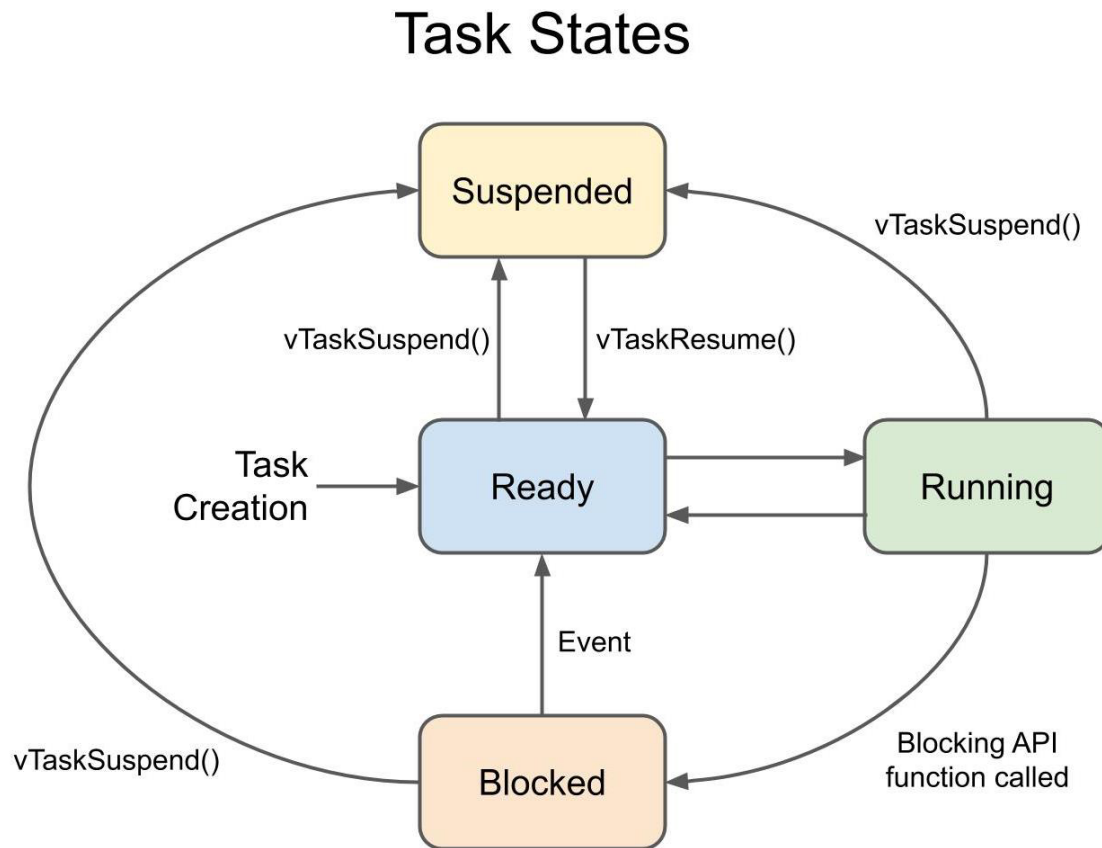
In short: FreeRTOS gives near-bare-metal efficiency with the safety net of real-time multitasking.

FreeRTOS Architecture Overview



FreeRTOS Tasks & Task States

A task is an independent thread of execution with its own stack and priority — the basic unit of concurrency in FreeRTOS.



The Four States

Ready — The task is capable of running (it isn't waiting on anything) but is *not* currently executing because a higher- or equal-priority task is using the CPU.

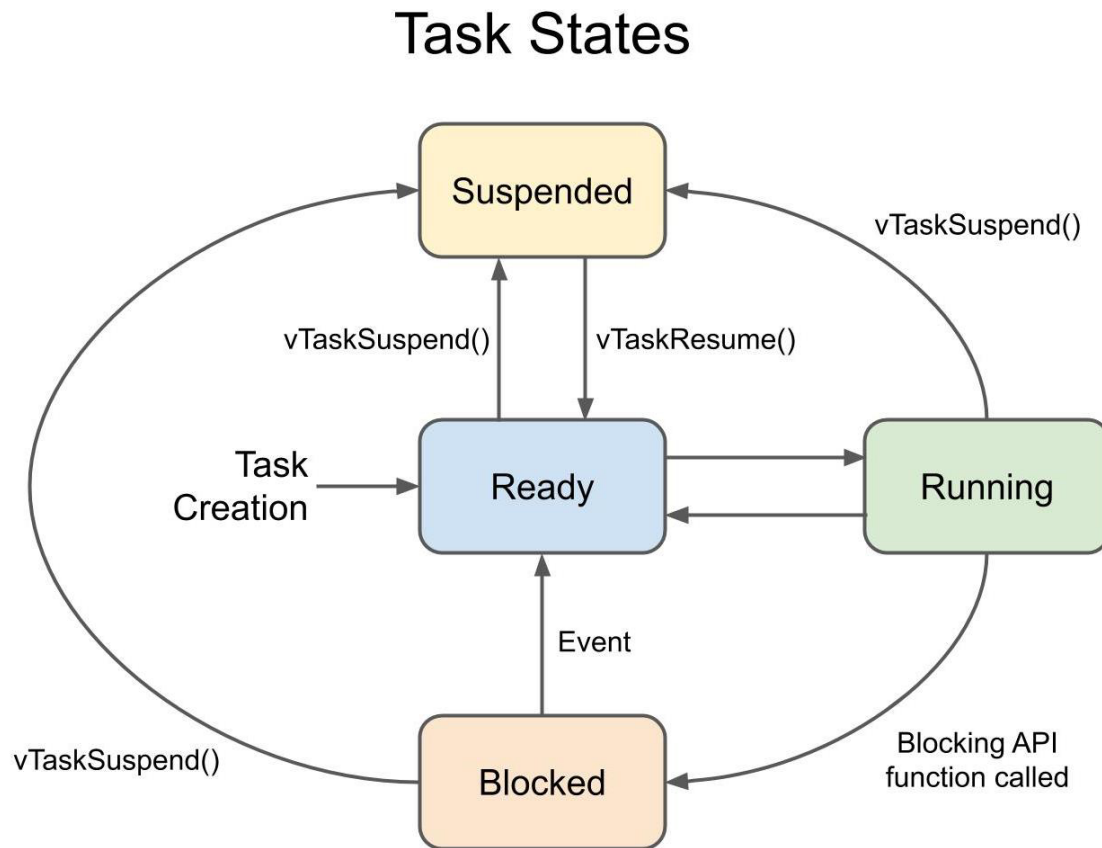
Running — The task is currently executing on the CPU. On a single-core system, only one task can be in this state at any instant.

Blocked — The task is waiting for something: a time delay to expire, a queue/semaphore to receive data, or another event. It's not eligible to run until that event occurs.

Suspended — The task is explicitly taken out of scheduling. It doesn't run and isn't considered by the scheduler at all — it only leaves this state when another task calls `vTaskResume()` on it.

FreeRTOS Tasks & Task States

A task is an independent thread of execution with its own stack and priority — the basic unit of concurrency in FreeRTOS.



Suspended is different from Blocked.

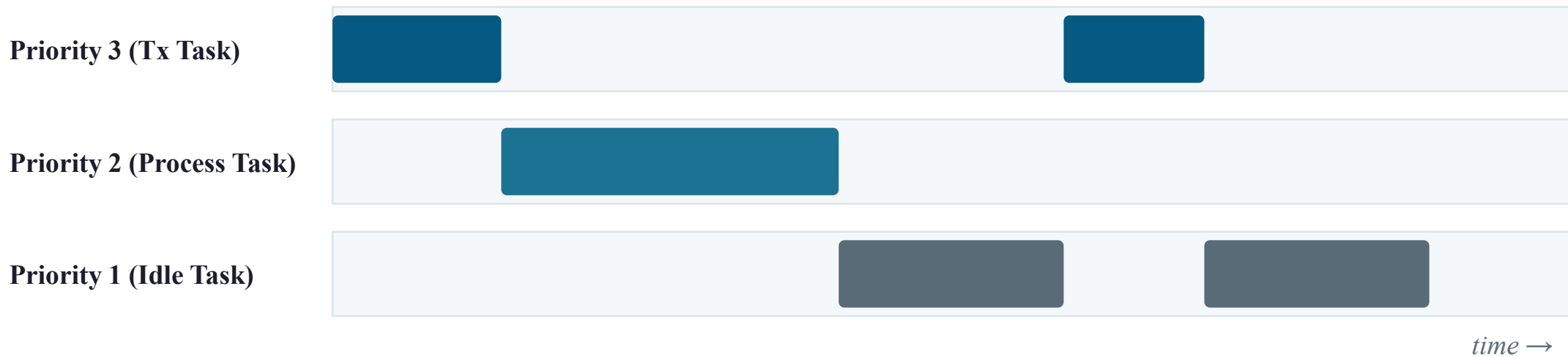
Blocked is *temporary and event-driven* — the kernel automatically wakes the task. Suspended is *explicit and manual* — only another task's `vTaskResume()` call can undo it, no matter how long it waits or what events occur.

Running is always singular (per core).

Every other Ready task, no matter how many exist, waits its turn — this is what the scheduler continuously arbitrates using task priority.

The FreeRTOS Scheduler: Priority-Based Preemption

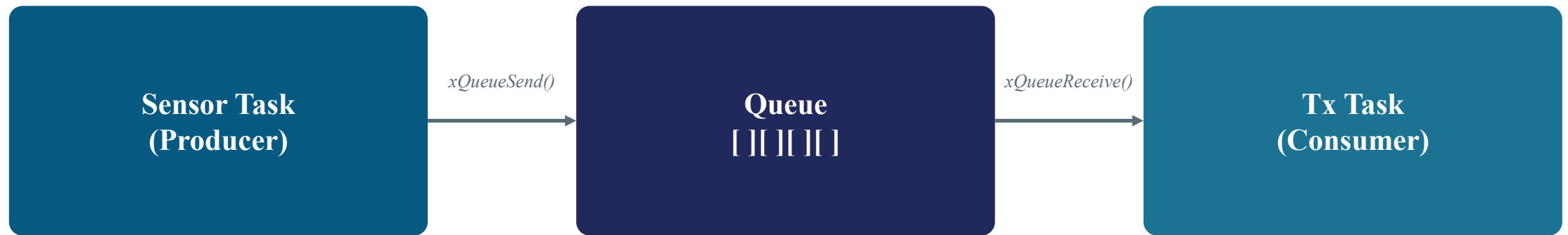
Rule: the highest-priority READY task always runs. Equal-priority tasks share the CPU via round-robin time slicing.



Preemption in action: the Process task (priority 2) runs until the higher-priority Tx task becomes ready at $t = 5.2$ — it immediately preempts, then Process resumes (or completes) once Tx finishes. The Idle task only runs when nothing else is ready.

Queues: Passing Data Safely Between Tasks

A queue is a FIFO buffer the kernel manages so tasks never touch shared memory directly — avoiding race conditions.



- Decouples producer and consumer — they can run at different speeds.
- A task calling `xQueueReceive()` on an empty queue automatically blocks (frees the CPU) until data arrives.
- This is exactly how a sensing task hands readings to a transmission task on the ESP32.

Semaphores & Mutexes

Binary Semaphore

A single-count flag used for signaling — e.g., an ISR signals “data ready” to unblock a task.

Counting Semaphore

Tracks how many of a limited resource are free — e.g., N available buffer slots.

Mutex

A binary semaphore with ownership + priority inheritance — protects a shared resource (e.g., a shared UART/I²C bus) from simultaneous access.

Priority inversion — worth knowing: if a low-priority task holds a mutex a high-priority task needs, the high-priority task is blocked behind it. A plain semaphore can leave this unresolved indefinitely; FreeRTOS mutexes solve it with priority inheritance, temporarily boosting the low-priority holder so it finishes and releases the resource quickly.

Putting It Together: Three Tasks on One ESP32

A typical IoT node runs several FreeRTOS tasks concurrently, each pinned to a priority that reflects how time-critical it is.

vSensorTask

Priority 2

Reads a sensor every 100 ms, pushes the value onto a queue, then blocks (vTaskDelay).

vProcessTask

Priority 1

Blocks on xQueueReceive(); filters/averages readings when data arrives.

vTxTask

Priority 3 (highest)

Woken by a binary semaphore every transmit interval; sends data over Wi-Fi/BLE — time-critical, so it preempts the others.

Without an RTOS, this would be one superloop juggling timers by hand. With FreeRTOS, each concern is an independent task — the scheduler guarantees the transmit deadline is met even while sensing and processing keep running.

Microcontroller + RTOS vs. SBC + Linux

Aspect	Microcontroller + RTOS	SBC + Linux
Example board	ESP32 / AVR (with FreeRTOS)	Raspberry Pi (with Linux)
CPU class	Single/dual-core MCU, tens–hundreds of MHz	Multi-core application processor, 1+ GHz
RAM	KBs – low MBs	512 MB – 8+ GB
Timing behavior	Deterministic — bounded worst-case latency	Best-effort — not guaranteed
Power draw	mW – low W; sleeps deeply between events	Watts; harder to deep-sleep
Best fit	Direct sensor/actuator control, battery nodes	Edge gateways, local AI inference, local dashboards

Where FreeRTOS Fits Among IoT Frameworks

Framework	Abstraction Level	Real-Time Capability	Footprint	Learning Curve
Arduino (IDE)	Very high (setup/loop)	None (superloop)	Small–Moderate	Gentle
MicroPython	High (Python)	Cooperative (asyncio)	Moderate–Large	Gentle
ESP-IDF + FreeRTOS	Low–Moderate (C, RTOS API)	Preemptive, real-time	Small	Steep
Bare-metal (no OS)	Lowest (direct registers)	Manual / none	Smallest	Steepest

Highlighted row: FreeRTOS trades ease-of-use for real-time guarantees — the right trade for production IoT nodes with timing constraints.

Key Takeaways

- An RTOS guarantees a bounded response time — not just fast execution, but on-time execution.
- FreeRTOS is preferred on constrained IoT devices for its tiny footprint, broad MCU support, and AWS-backed ecosystem.
- Tasks, the priority-based preemptive scheduler, queues, and semaphores/mutexes are the four building blocks that let one MCU safely multitask.
- Choose the platform to the job: RTOS + microcontroller for deterministic, low-power control; Linux + SBC for compute-heavy edge processing.